// TECHNICAL BLUEPRINT - IMPLEMENTATION REFERENCE

Engineering Blueprint for the Legal Workflow OS

Platform layers - Service contracts - Workflow runtime - Deployment operations

This blueprint converts the platform strategy into implementation-ready technical decisions. It defines architecture boundaries, canonical entities, connector contracts, workflow states, tenancy and access guarantees, and pilot deployment operations for Phase 1 through Phase 4.

MAJOR SECTIONS	CORE ENTITIES	DIAGRAM FIGURES	PLATFORM LAYERS
10	10	8	L1--L5

PREPARED BY

Vincent Nyamao

PREPARED FOR

CEO, Esheria

DOCUMENT VERSION

v1.0 - March 2026

CLASSIFICATION

Confidential - Engineering Review

// BLUEPRINT SUMMARY

Implementation Scope and Intent

This technical blueprint is the implementation bridge between product strategy and engineering execution. It provides one canonical reference for architecture boundaries, data contracts, workflow runtime behavior, tenancy isolation, connector integration, and operational readiness. The purpose is to reduce ambiguity before build and ensure all teams are implementing the same system, not parallel interpretations of the system.

> // HOW TO USE THIS DOCUMENT
>
> Sections 01 to 04 define architecture and runtime contracts. Section 05 defines access and tenant security controls. Sections 06 to 08 define operations, deployment, and service integration rules. Use the final implementation guidance as a pre-pilot acceptance checklist.

01 System Architecture

Five-layer stack and module boundaries

Architecture Overview

The platform is organized into five layers with strict adjacency dependencies. No layer may call or depend on a non-adjacent layer in production code. This is the principal guardrail that keeps orchestration logic stable as more workflows and connectors are introduced.

Layer	Name	Canonical Prefix	Responsibility
L5	**Experience**	`/` routing	Institution Workspace, Operator Console, Public Surface.
L4	**Control Plane**	`/api/workflows/*`	Workflow orchestration, state transitions, approvals, SLAs.
L3	**Object Model**	`schema.*`	Canonical entities, tenancy keys, and lifecycle invariants.
L2	**Connector Gateway**	`/internal/*`	Service mediation, auth delegation, retries, normalized errors.
L1	**Capability Services**	service-native	OCR, contract, research, drafting, analytics services.

Architecture Acceptance Criteria

- Every workflow transition is emitted into immutable audit storage.
- Every connector operation has timeout, retry, and error classification behavior.
- Every object write is schema validated before persistence.
- No service in L1 is directly callable by front-end clients.

> `// PHASE 1 EXIT GATE`
>
> The schema and layer-boundary review is the mandatory gate before MVP coding. Build begins only after explicit sign-off from platform lead and service owners.

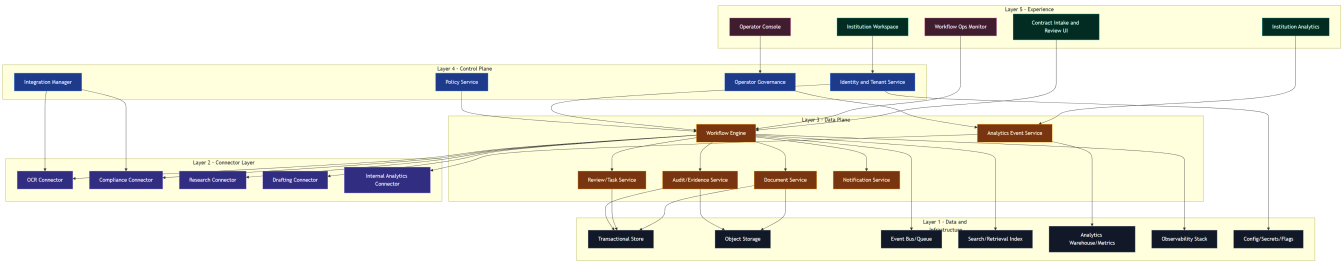

Figure 1: *

Figure 01 - Layered architecture and boundary controls

02 API Boundaries

External and internal API separation

Boundary Contract

Two API zones exist and must never be conflated: external product APIs and internal service APIs.

Prefix	Caller	Auth	Purpose
`/api/*`	**Web clients and approved external integrations**	org JWT	Product and workflow operations.
`/internal/*`	**Connector Gateway only**	service token/mTLS	Calls from platform to capability services.

Versioning and Error Model

- Product APIs are versioned (`/api/v1/*`) with backward-compatible evolution by default.
- Internal connector contracts are pinned by adapter version in the connector registry.
- Platform error classes: `AUTH_ERROR`, `VALIDATION_ERROR`, `RATE_LIMITED`, `TIMEOUT`, `SERVICE_UNAVAILABLE`, `INTERNAL_ERRO`

Security Control	Enforcement
Token validation	Issuer, expiry, org scope, and endpoint scope checked at gateway.
Per-tenant rate limit	Burst and sustained limits applied before workflow entry.
Request tracing	`request_id` propagated into actions and audit events.
Secrets rotation	Connector credentials rotated every 90 days.

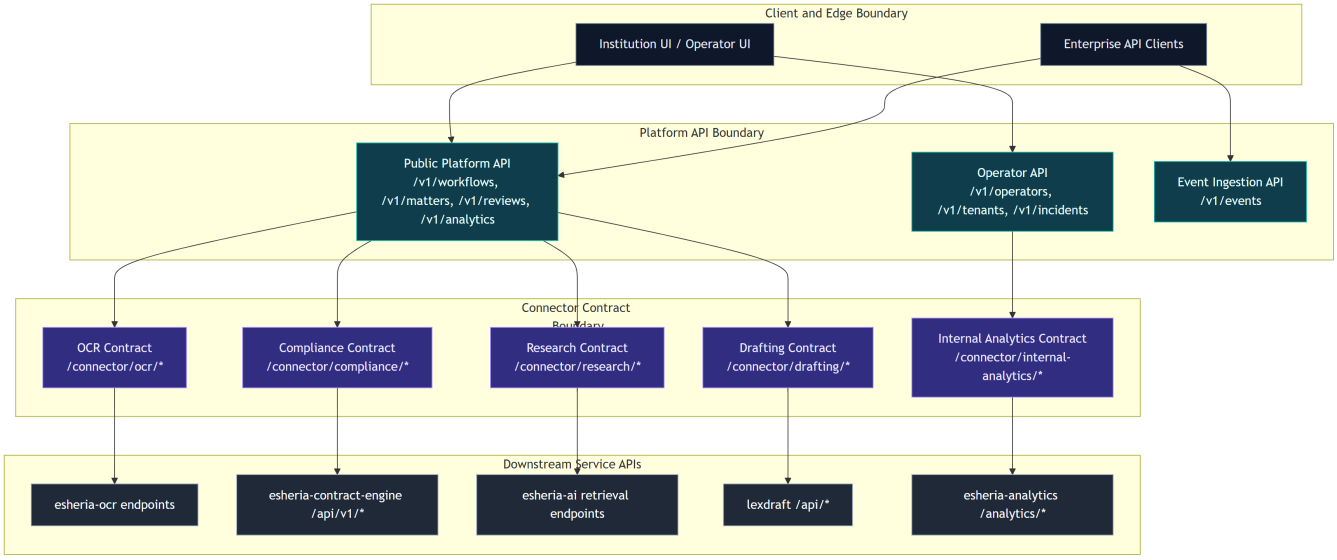

Figure 2: *

Figure 02 - External API and internal connector boundary map

03 Canonical Ontology Model

Shared objects and schema governance

Entity Registry

Entity	ID	Purpose	Key Fields
`Organization`	E1	Tenant root object for isolation and policy scope.	`org_id,name,sector,policy_set_id`
`User`	E2	Identity scoped to an organization.	`user_id,org_id,role,permissions[]`
`Matter`	E3	Business context for contracts and reviews.	`matter_id,org_id,type,status,owne`
`Document`	E4	Source contract and processing status.	`doc_id,matter_id,source,ocr_statu`
`Clause`	E5	Extracted legal clause artifact.	`clause_id,doc_id,clause_type,text`
`Risk`	E6	Compliance or policy finding.	`risk_id,clause_id,severity,policy`
`WorkflowRun`	E7	Runtime execution envelope.	`run_id,matter_id,workflow,current`
`Action`	E8	Atomic step execution record.	`action_id,run_id,actor,input,outp`
`Review`	E9	Human reviewer decision and rationale.	`review_id,doc_id,reviewer_id,deci`
`Citation`	E10	Evidence linkage for explainability.	`citation_id,action_id,source_uri,`

Schema Governance Rules

- Every object except `Organization` must carry `org_id` and be protected by RLS.
- Additive changes are allowed in major version; destructive changes require migration cycle.
- Action and audit records are append-only and cannot be mutated in place.

```
// MIGRATION DISCIPLINE
```

Every migration pull request must include forward SQL, rollback strategy, compatibility impact, and data backfill notes. Migrations without rollback are rejected.

Retention Policy Snapshot

Object Class	Retention	Rationale
Audit events	7 years	Regulatory traceability and incident forensics.
Workflow actions	5 years	Operational analysis and model quality inspection.
Source documents	Policy based	Jurisdiction and enterprise contract obligations.
Vector embeddings	12 months default	Cost/performance balance with re-index option.

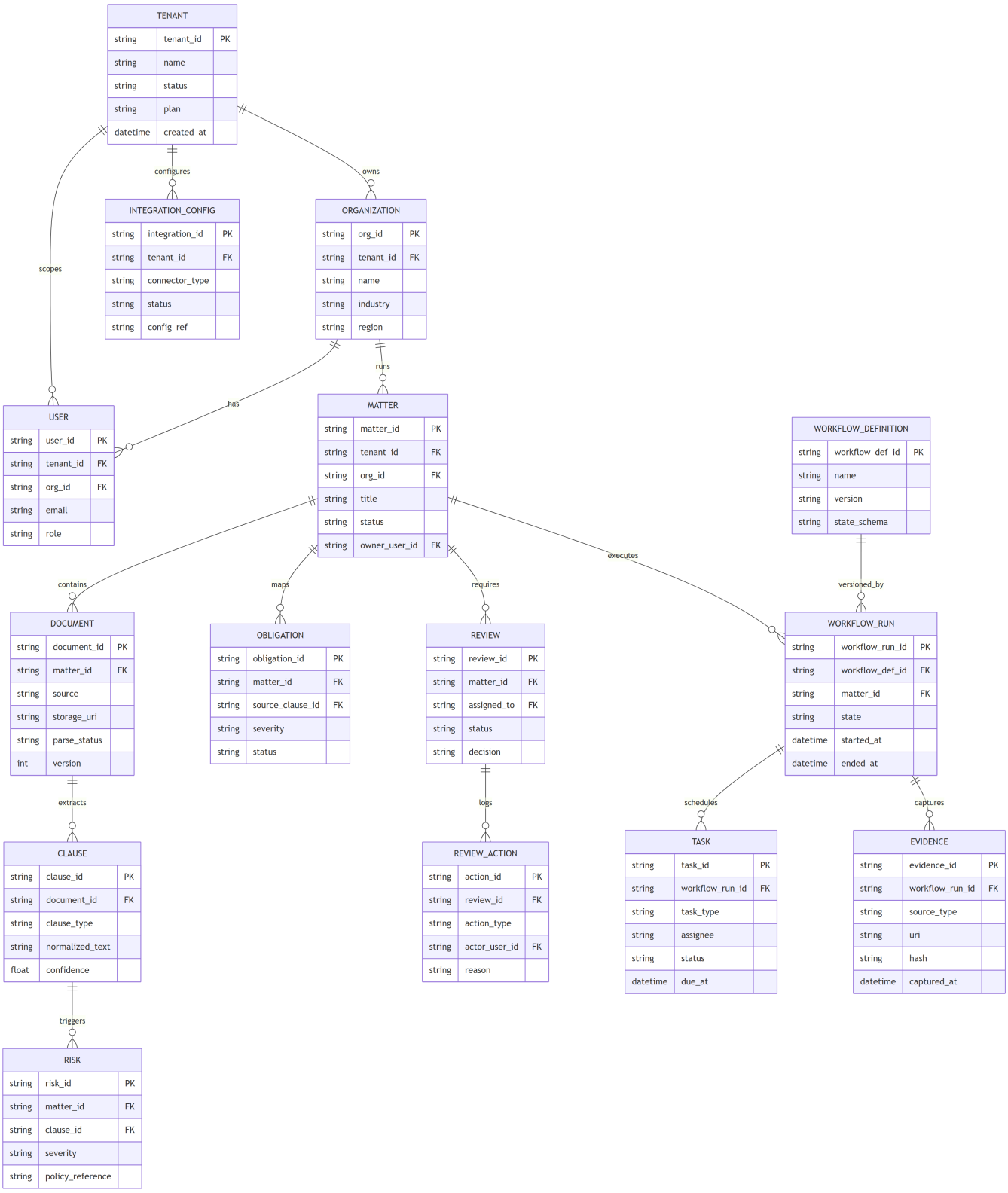

Figure 3: *

Figure 03 - Canonical entity relationship model

04 Workflow Engine Design

Deterministic transitions and human review loop

Runtime Principles

- Deterministic transition rules for every branch and terminal state.
- Idempotent operators keyed by `run_id + step_id`.
- Human review as a first-class runtime state, not a side callback.
- SLA deadlines and escalations managed by scheduler with immutable trail.

Lifecycle State Model

State	Description	Valid Transitions
PENDING	Workflow run created and validated.	-> RUNNING
RUNNING	Current step executing in connector/service layer.	-> AWAITING_REVIEW, FAILED
AWAITING_REVIEW	Waiting for human decision.	-> APPROVED, REJECTED, ESCALATED
ESCALATED	SLA threshold exceeded and routed upward.	-> AWAITING_REVIEW
APPROVED	Human accepted recommendation.	-> COMPLETED
REJECTED	Human rejected and remediation loop starts.	-> RUNNING
COMPLETED	All steps done and analytics emitted.	terminal
FAILED	Retry budget exceeded, manual intervention needed.	-> RUNNING

Operator Registry for Contract Intake

Operator	Type	Input	Output
`IngestionOperator`	AI	`doc_id`	Parsed text and classification
`ExtractionOperator`	AI	Parsed text	Structured clauses
`ComplianceOperator`	AI	Clauses and policy set	Risk findings with severity
`ResearchOperator`	AI	Risks and clauses	Evidence citations
`DraftingOperator`	AI	Risks and citations	Alternative language proposals
`ReviewOperator`	AI	Aggregate context	Review briefing package
`HumanReviewer`	Human	Briefing package	Decision and notes
`ApprovalRouter`	System	Reviewer decision	Final branch and close path

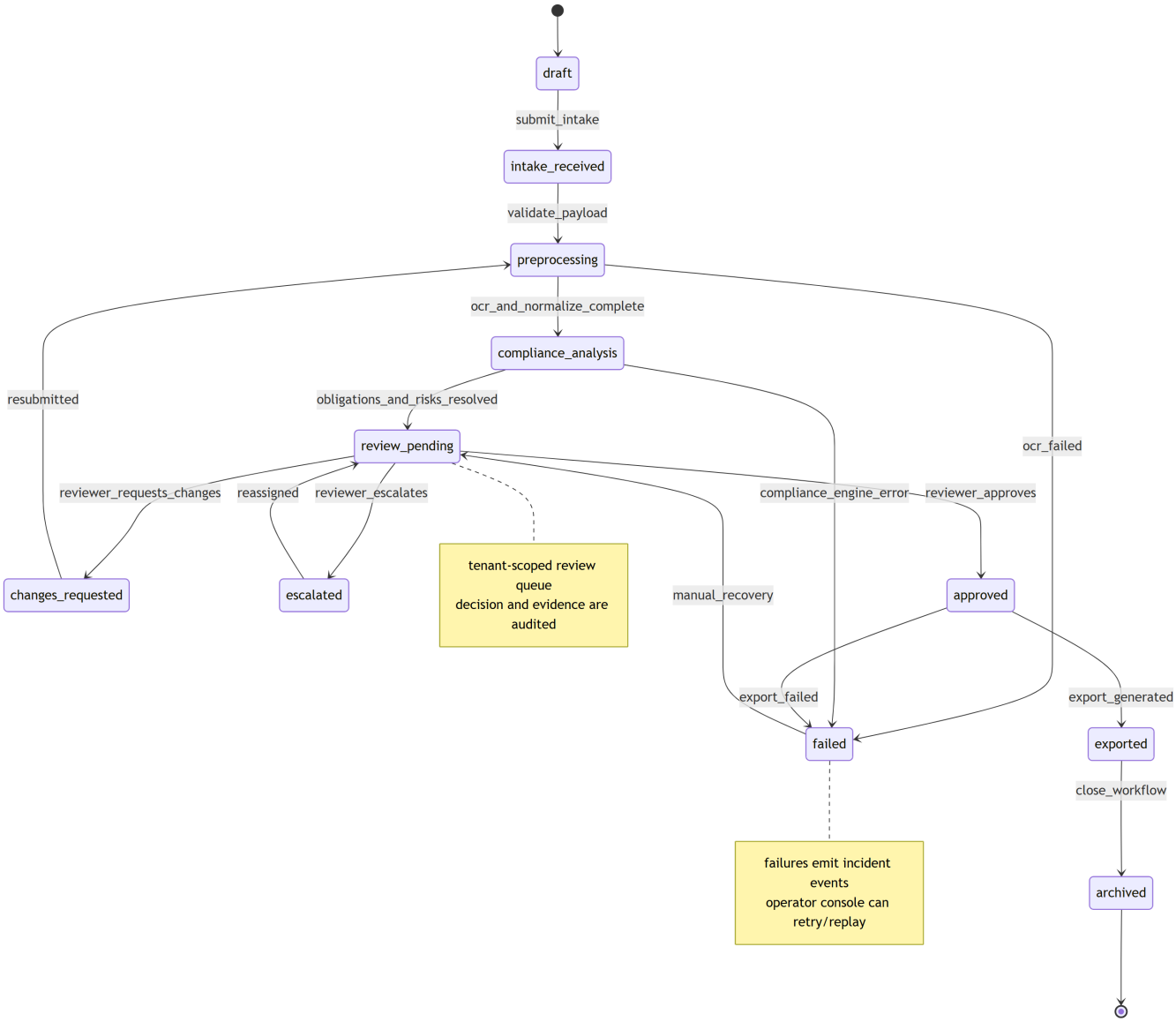

Figure 4: *

Figure 04 - Workflow state machine and operator execution path

05 Tenancy and Auth

Isolation, RBAC, and analytics separation

Tenancy Architecture

The MVP uses logical multi-tenancy on shared infrastructure with strict database-enforced isolation. Application filtering is defense-in-depth only. Primary isolation is row-level policy enforcement.

JWT Contract

Claim	Description
`sub`	User identity key.
`org`	Tenant key used for authorization and query scoping.
`role`	Reviewer/Admin/Operator/Super Admin/Auditor role mapping.
`scopes[]`	Endpoint permission set.
`exp`	Token expiration timestamp.
`iss`	Trusted issuer identity.

Role Matrix

Role	Surface	Capabilities
`reviewer`	Institution	Review assignments, approve/reject, annotate findings.
`admin`	Institution	User management, policy config, integration setup.
`api_client`	Institution API	Machine access on explicit scoped grants only.
`operator`	Internal console	Monitor runs, triage failures, trigger retries.
`super_admin`	Internal console	Controlled emergency actions with audit reason.
`auditor`	Internal console	Read-only audit and event visibility.

// OPERATOR CONSOLE ISOLATION

Operator Console must run on separate subdomain and issuer from institution workspace. No institution account can escalate by URL manipulation or role mutation.

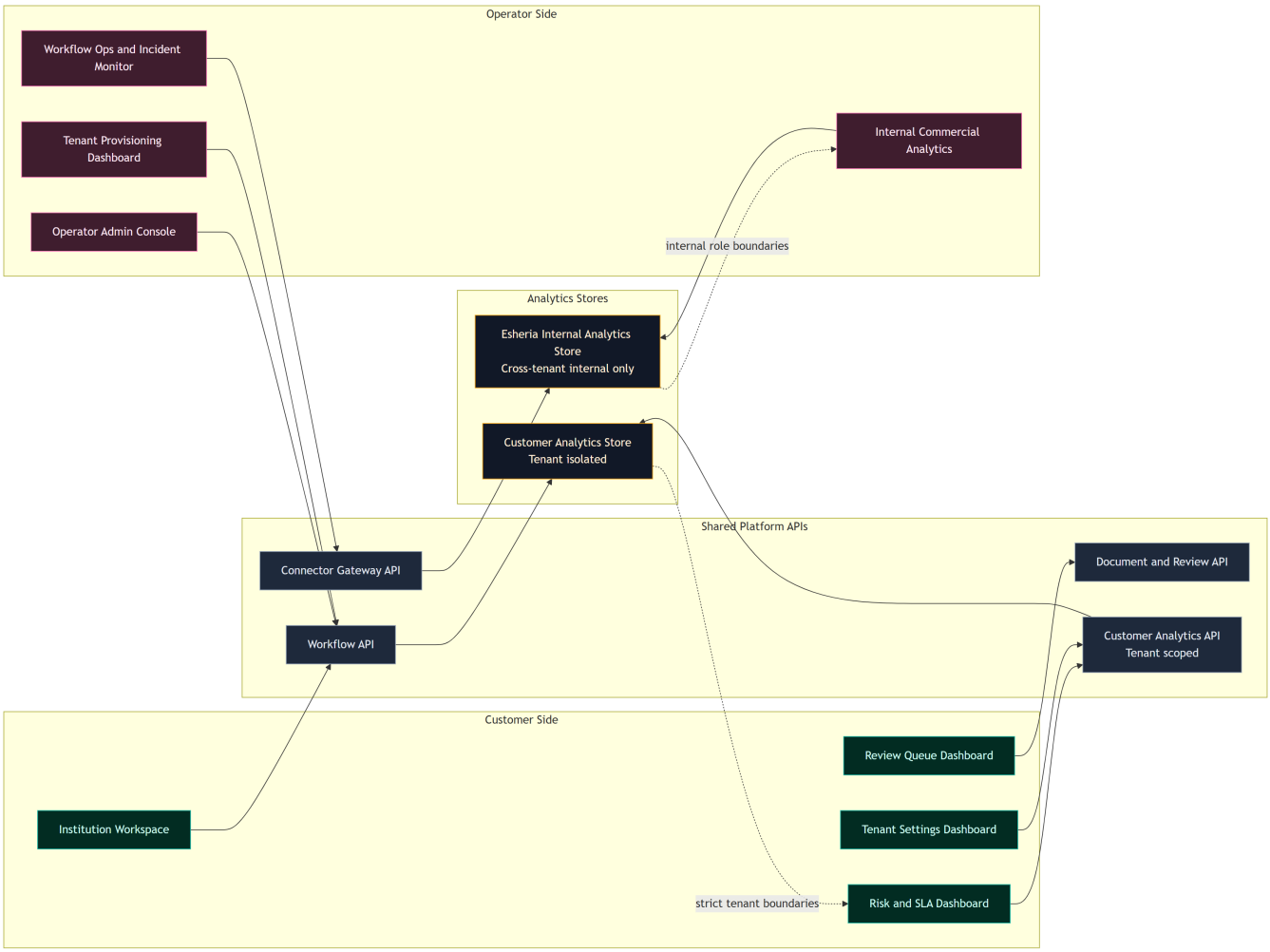

Figure 5: *

Figure 05 - Institution and internal analytics separation

06

Control vs Data Plane

Orchestration boundaries and execution rules

Responsibility Split

Concern	Control Plane (L4)	Data Plane (L2+L1)
Primary role	Workflow orchestration and policy enforcement	Capability execution and data extraction
State ownership	Owns `WorkflowRun` transitions	Emits step outputs and service metrics
Routing decisions	Determines next step, retries, and escalation path	No direct routing authority
Audit behavior	Emits immutable transition events	Emits execution logs and telemetry
SLA management	Deadlines, escalation scheduling, alerts	No SLA ownership
Retry strategy	Policy by step class	Runtime retry mechanics by connector

Boundary Controls

- Service adapters cannot mutate workflow state directly.
- Connector gateway cannot contain business decision logic.
- Emergency force-advance requires dual approval with reason capture.

> // GOVERNANCE CONTROL
>
> Any pull request introducing cross-plane leakage is blocked pending architecture review.

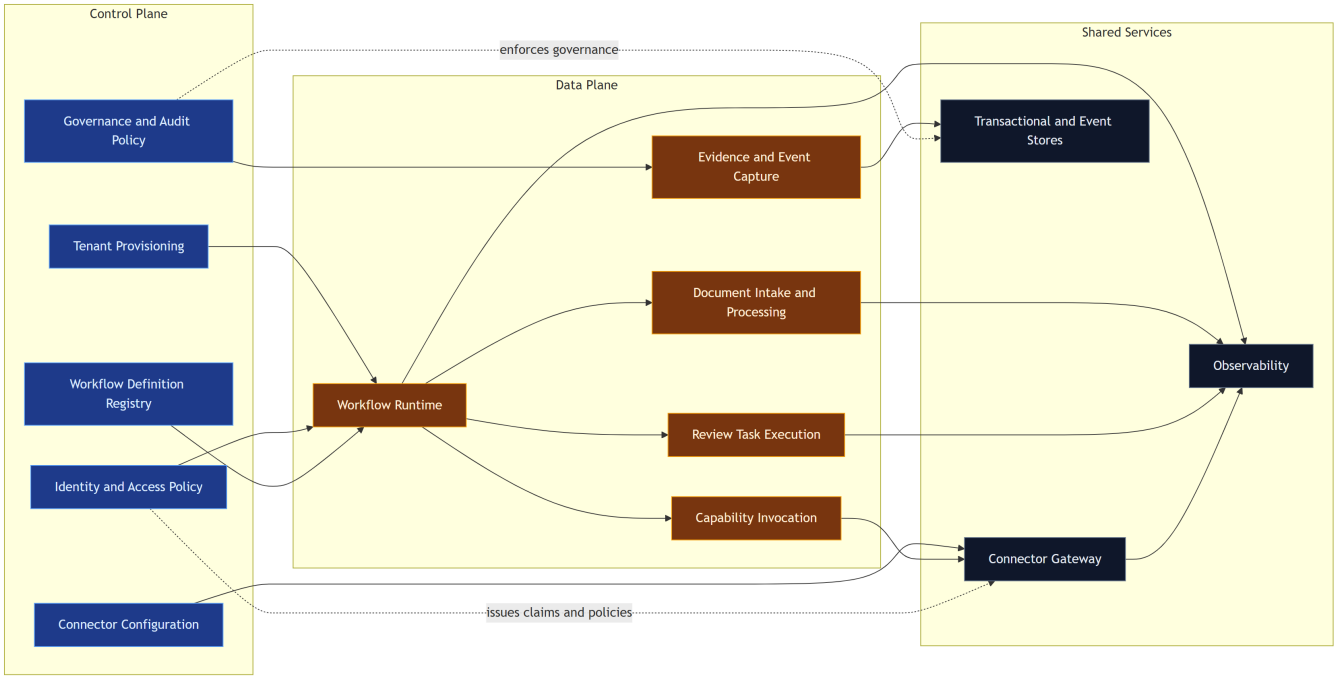

Figure 6: *

Figure 06 - Control and data plane split with boundary enforcement

07 Deployment Topology

MVP runtime stack and reliability targets

Infrastructure Components

Component	Technology	Responsibility
API Gateway	`nginx/ALB`	TLS termination, ingress control, per-tenant throttling
Auth Service	`JWT issuer`	token issue, refresh, revocation
Platform Backend	`Node.js/Go`	workflow engine, connectors, external API
Message Queue	`Redis Streams`	async step execution and buffering
Primary Database	`PostgreSQL + RLS`	canonical object persistence and isolation
Object Storage	`S3-compatible`	source documents and generated artifacts
Vector Index	`pgvector/Pinecone`	semantic retrieval for research stage
Observability	`Prometheus/Grafana`	metrics, alerts, and health dashboards

Reliability Targets

Metric	Target	Measurement Scope
Platform API availability	99.5% monthly	Success ratio on `/api/*` requests
Workflow completion success	98%	Contract Intake runs with no manual recovery
P95 connector latency	< 2.0s	Excluding long OCR operations
SLA breach rate	< 5%	Review steps exceeding deadline windows
MTTR (critical incidents)	< 45 min	Detection to recovery for P1 incidents

Required Runbooks Before Pilot

1. Incident triage and connector isolation.
2. Failed workflow recovery and replay.
3. Maintenance mode and service drain operations.
4. Data retention and legal hold execution.

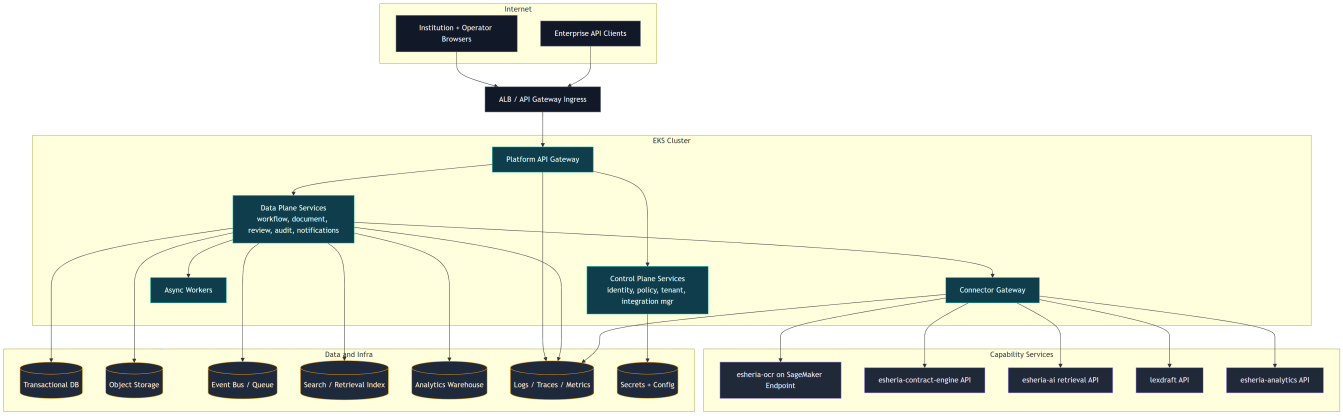

Figure 7: *

Figure 07 - MVP deployment topology and operational boundaries

08

Integration Architecture

Connector contracts and rollout sequencing

Connector Contract

Contract Element	Requirement
Connector identity	Unique id, target service name, and status lifecycle.
Call behavior	Endpoint call with timeout budget and structured result payload.
Error normalization	Service-specific errors mapped to platform error classes.
Health endpoint	Periodic health and latency report for monitoring plane.
Metadata output	Includes service version and timing on every response.

Connector Rollout Registry

Connector	Service	MVP Status	Delivery Notes
`ocr-connector`	`esheria-ocr`	Active	First build to validate gateway control path.
`contract-connector`	`contract-engine`	Active	Core extraction and policy risk scoring.
`research-connector`	`esheria-ai`	Active	Citation evidence chain for reviewer confidence.
`drafting-connector`	`lexdraft`	Planned	Stub in P2, complete in P3.
`analytics-connector`	`esheria-analytics`	Planned	Emit-only in P2, full query model in P3.

Phase Plan Snapshot

Phase	Name	Duration	Output
P0	**Alignment**	2–3 wks	Scope lock, success metrics, and ownership model.
P1	**Design**	3–4 wks	Schema lock, API contracts, workflow spec.
P2	**MVP Build**	6–8 wks	End-to-end contract intake and review run.
P3	**Internal Pilot**	3–4 wks	Reliability hardening and operational readiness.
P4	**B2B Pilot**	4–6 wks	Institution pilot package and reporting baseline.

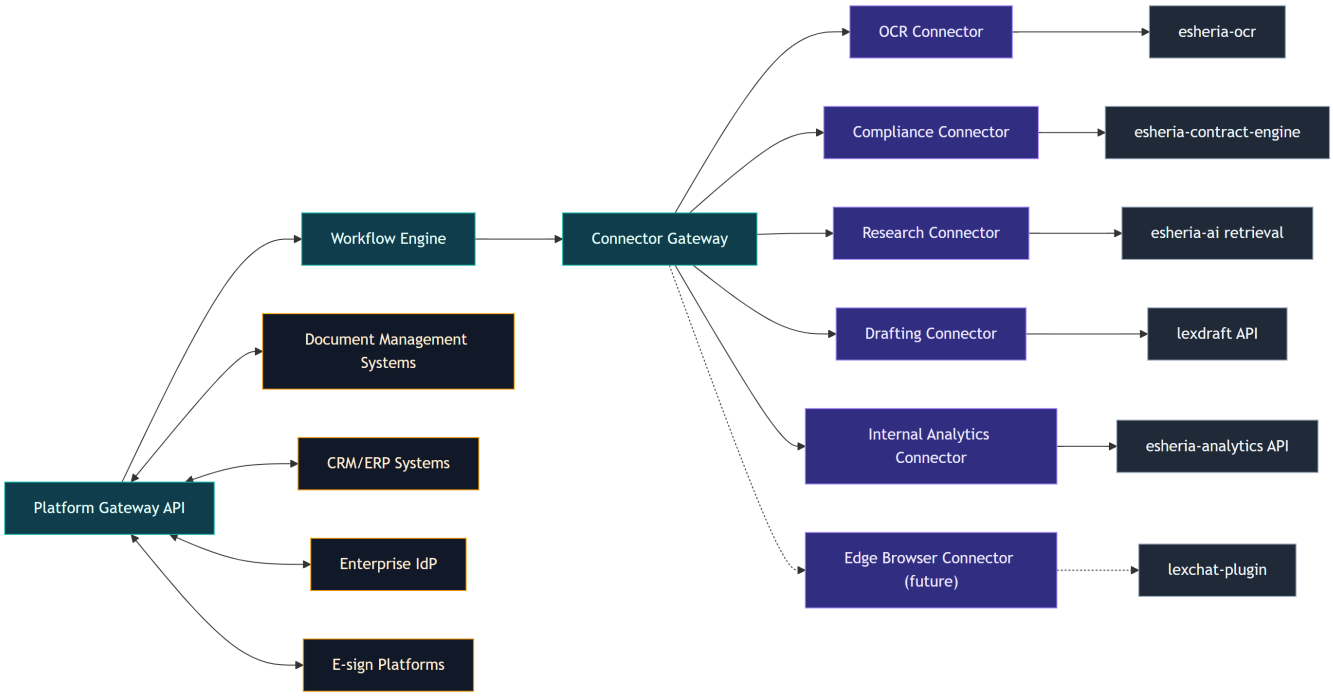

Figure 8: *

Figure 08 - Connector mediation across capability services

// IMPLEMENTATION GUIDANCE

Engineering Build Checklist

Mandatory Sequence

1. Lock canonical schema and RLS policies.
2. Implement workflow engine with deterministic transitions.
3. Build connector gateway with timeout, retry, and normalized errors.
4. Deliver OCR and contract connectors, then research connector.
5. Ship reviewer workspace with full audit timeline.
6. Run full pilot script and incident simulation tests.

MVP Definition of Done

- One contract completes all planned workflow stages with human decision closure.
- Audit chain is queryable by `run_id` in under 2 seconds.
- Tenant isolation tests pass at gateway, API, and database levels.
- Failure injection proves graceful degradation and deterministic recovery.

> // SCHEMA FIRST
>
> The highest risk in platform delivery is writing service logic before schema consensus. Keep feature work in stub mode until schema and migration policy are formally approved.

Related Artifacts

- `artifacts/documents/executive_proposal.tex`
- `docs/discovery/repo-deep-scan.md`
- `docs/discovery/api-catalog.md`
- `docs/architecture/consistency-report.md`

09

Quality Gates

Engineering, security, and reliability release controls

Pre-Pilot Quality Gate Matrix

Gate	Status Rule	Verification Evidence
Schema governance	Must pass	Signed schema review and migration rollback plan.
Tenant isolation	Must pass	RLS tests across positive and negative org scenarios.
Workflow determinism	Must pass	Replay tests proving identical branch outcomes.
Connector resilience	Must pass	Timeout, retry, and circuit-breaker simulation logs.
Audit completeness	Must pass	End-to-end run with full state and action timeline.
Security baseline	Must pass	Token validation, scope checks, and secret rotation checks.

Test Coverage Expectations

- Unit coverage for workflow transition logic and connector error mapping.
- Integration coverage for all active connectors in the Contract Intake path.
- Security coverage for auth bypass attempts and cross-tenant access attempts.
- Chaos-style checks for service timeout, partial outage, and restart recovery.

// RELEASE RULE

No pilot release is approved unless all quality gates are green and one full dry-run demo is completed with incident simulation included.

10

Risk Register

Top technical risks and mitigations for MVP delivery

Delivery Risk Register

ID	Risk	Severity	Mitigation
R1	**Schema churn during build**	High	Freeze entity contract at P1 exit and enforce change-control board.
R2	**Cross-layer coupling**	High	Architecture review checks in pull request policy.
R3	**Connector instability**	Medium	Staged rollout, circuit breaker thresholds, replay queue fallback.
R4	**Human review SLA misses**	Medium	Escalation scheduling with dual-threshold alerts and load balancing.
R5	**Incomplete audit trace**	High	Mandatory event emission on every transition and reconciliation job.
R6	**Multi-tenant data leakage**	Critical	RLS-first model, negative tests in CI, and canary monitoring.

Post-Pilot Expansion Priorities

- SSO and enterprise identity federation hardening.
- Additional workflow templates beyond contract intake.
- Expanded connector library for DMS, CRM, and ERP targets.
- Capacity planning for multi-region failover and disaster recovery.

// PILOT DISCIPLINE

Pilot success depends on scope control. Do not expand workflow variety before reliability, audit integrity, and tenant isolation metrics meet target thresholds for two consecutive cycles.